TRUSTED STUDENT HOUSING

TSH

CAHIER DES CHARGES

Application web de logement étudiant, colocation, meubles et déménagement

Architecture microservices (8 services) — Modèle communautaire unifié

13 novembre 2026

Sommaire

1. Présentation du projet
2. Problématique et solutions
3. Objectifs
4. Acteurs et rôles
5. Découpage en microservices
6. Architecture globale
7. Diagramme de classes
8. Exigences fonctionnelles par microservice
9. Exigences non fonctionnelles
10. Contrats d'API principaux
11. Stack technique

1. Présentation du projet

Plateforme centralisée facilitant la recherche de logements étudiants, la colocation, l'achat/vente de meubles d'occasion et la mise en relation avec des transporteurs.

Élément	Détail
Nom	Trusted Student Housing (TSH)
Domaine	Logement étudiant, colocation, marketplace de meubles, déménagement
Zone géographique	Tunisie (couverture nationale)
Type d'application	Web responsive en architecture microservices autonomes
Modèle d'accès	Communautaire unifié (un compte unique accède à tous les modules)
Langue	Français / Anglais
Devise	Dinar tunisien (DT / TND)

---

2. Problématique et solutions

2.1 Problèmes identifiés

- **Annonces non fiables :** Logements obsolètes, déjà loués, surévalués ou profils non identifiés.
- **Colocation au hasard :** Difficulté de trouver des colocataires compatibles en budget et mode de vie.
- **Accords verbaux :** Absence de formalisme, aucun contrat écrit ni reçu de loyer exploitable.
- **Échanges dispersés :** Discussions téléphoniques informelles sans historique ni traçabilité.
- **Installation complexe :** Difficulté de meubler un logement à bas coût et de trouver un déménageur fiable.

2.2 Solutions apportées par TSH

- **Numéro vérifié (OTP SMS) :** Un numéro unique vérifié par compte pour bannir les faux profils.
- **Matching intelligent :** Algorithme de compatibilité selon budget, école et préférences.
- **Formalisation documentaire :** Génération automatique de contrats et reçus de paiement en PDF.
- **Messagerie & Notifications :** Chat temps réel (WebSocket) et notifications d'événements (Kafka).
- **Modules complémentaires :** Marketplace de meubles entre étudiants et recherche de transporteurs avec devis.

3. Objectifs

Objectifs métier du projet avec association directe des objectifs Data Science correspondants.

Objectif métier	Contribution attendue	Objectif Data Science
Renforcer la confiance	Authentification par OTP SMS unique et traçabilité des comptes.	
Faciliter l'accès au logement	Recherche multicritère, annonces complètes et photos certifiées.	Estimation de loyer : fourchette de prix indicative selon la zone et le type de bien.
Faciliter la colocation	Mise en relation d'étudiants et gestion des demandes.	Matching de colocation : score de compatibilité selon budget, établissement et préférences.
Sécuriser le parcours locatif	Génération de contrats PDF, suivi des paiements en ligne et reçus téléchargeables.	
Centraliser les échanges	Messagerie temps réel interne et alertes sur événements applicatifs.	
Équiper le logement	Marketplace de meubles d'occasion (achat, vente, don, échange).	
Simplifier le déménagement	Mise en relation avec des transporteurs, comparaison de devis et réservation.	

4. Acteurs et rôles

Modèle communautaire unifié : tout utilisateur inscrit dispose de l'ensemble des droits métiers.

Rôle Keycloak	Acteur	Responsabilités
ROLE_USER	Utilisateur unique	Accès à tous les modules : publier ou chercher un logement, chercher un colocataire, vendre/acheter des meubles, proposer ou réserver un transport, chatter et payer.
ROLE_ADMIN	Administrateur	Supervision globale de la plateforme, modération des annonces (logements et meubles), traitement des réclamations et blocage des comptes frauduleux.

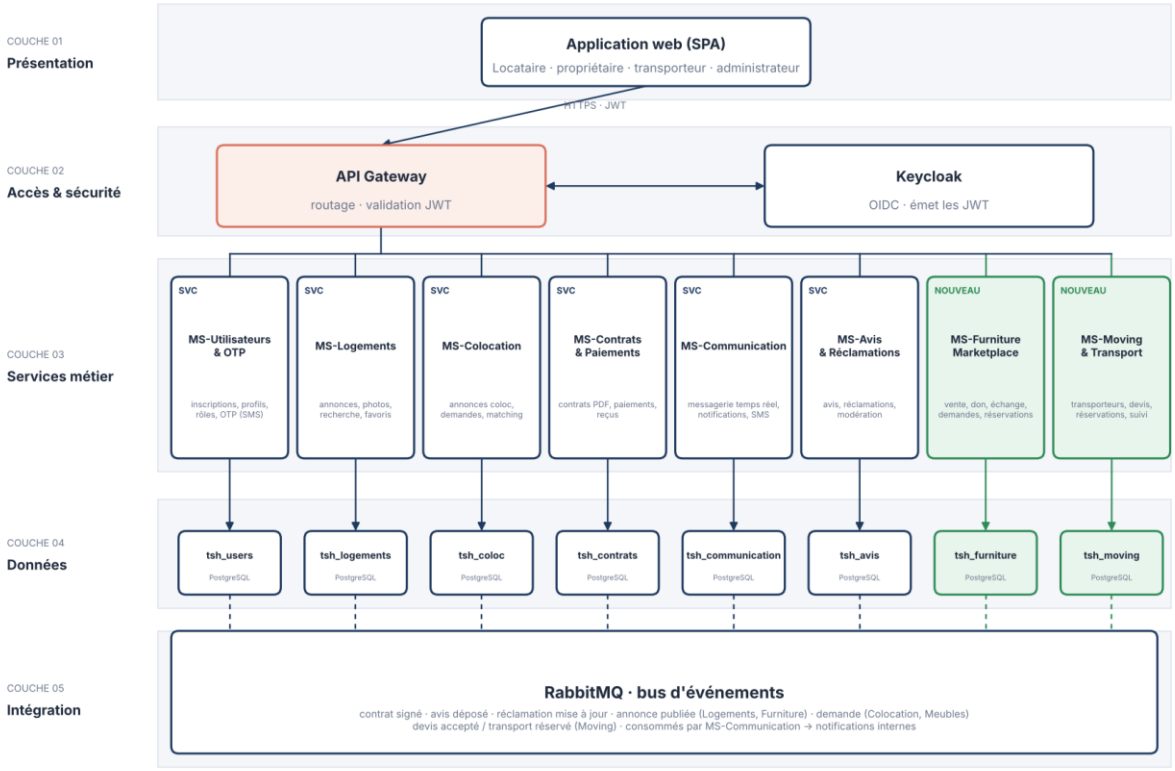
5. Découpage en microservices

Architecture en huit microservices autonomes avec base de données isolée (Database-per-Service).

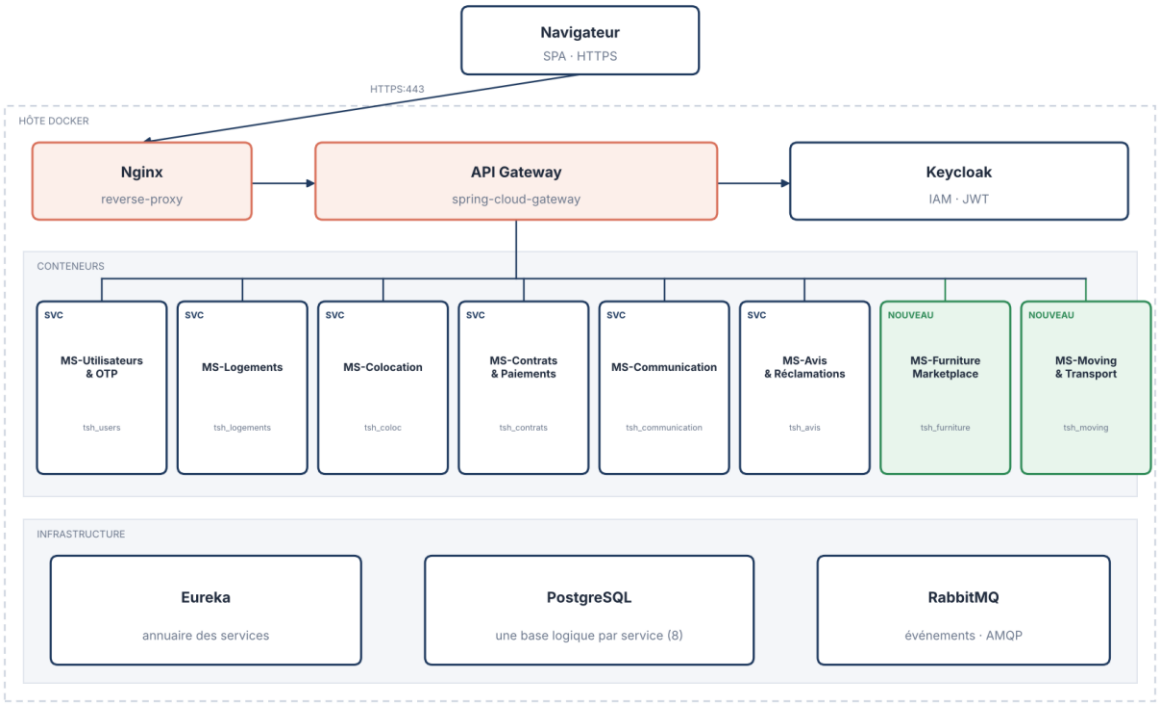
N°	Microservice	Responsabilités principales	Base logique
1	MS-Utilisateurs & OTP	Gestion des comptes, vérification OTP SMS, rôles et intégration Keycloak.	tsh_users
2	MS-Logements	CRUD annonces logements, recherche multicritère, favoris et photos.	tsh_logements
3	MS-Colocation & Matching	Annonces de colocation, demandes de contact et matching de profils.	tsh_coloc
4	MS-Contrats & Paiements	Génération des contrats PDF, paiements en ligne et reçus.	tsh_contrats
5	MS-Communication	Messagerie temps réel WebSocket et notifications Kafka.	tsh_communication
6	MS-Avis & Réclamations	Dépôt d'avis, notes étoilées et traitement des réclamations.	tsh_avis
7	MS-Furniture Marketplace	Annonces meubles (vente/don/échange), recherche, photos et réservations.	tsh_furniture
8	MS-Moving & Transport	Profils transporteurs, demandes de déménagement, devis et réservations.	tsh_moving

6. Architecture globale

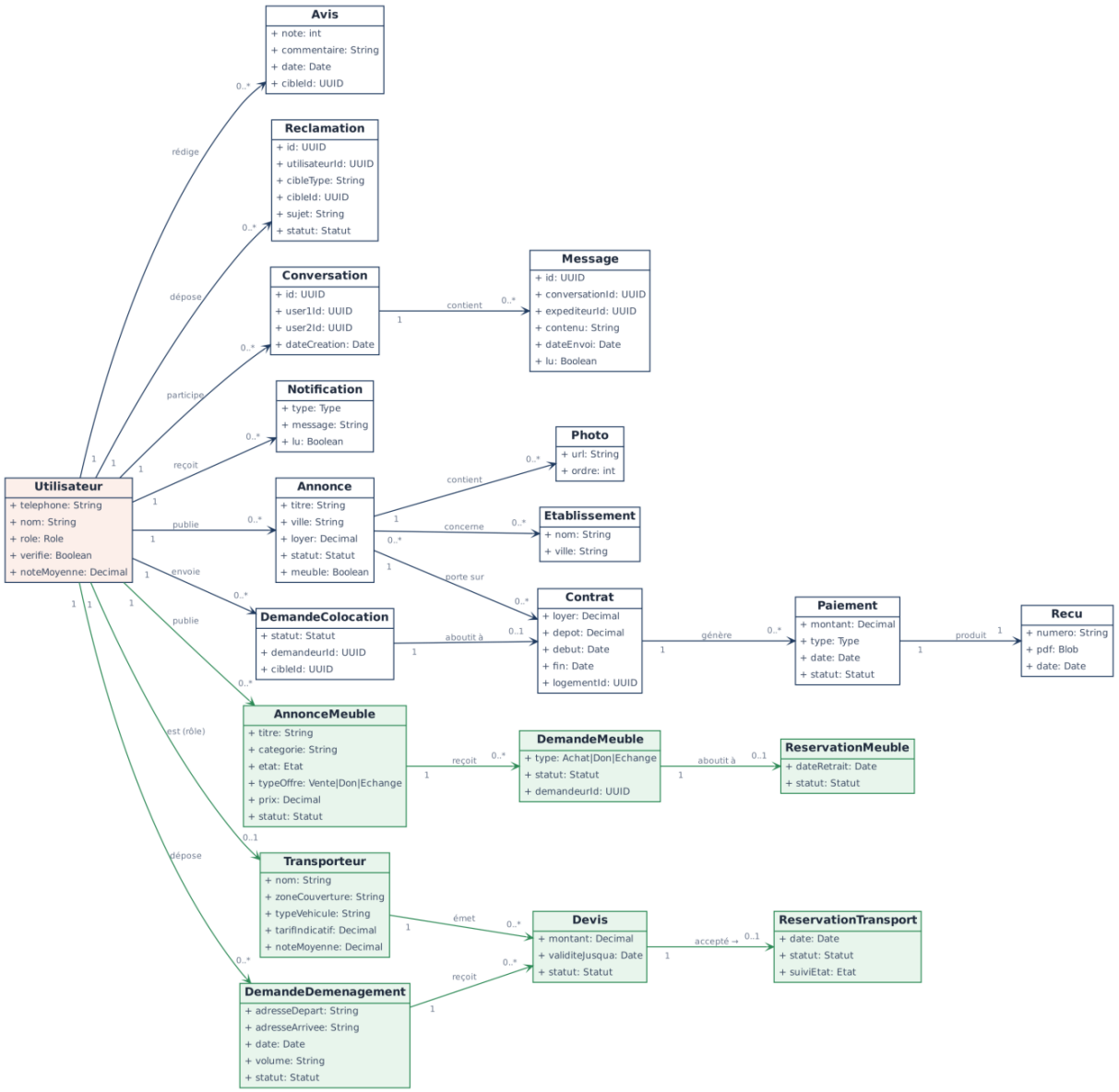
6.1 Architecture logique



6.2 Architecture physique et déploiement



7. Diagramme de classes



8. Exigences fonctionnelles par microservice

Chaque microservice dispose de ses exigences autonomes pour le développement et la validation.

8.1 MS-Utilisateurs & Authentification

ID	Exigence fonctionnelle
AUTH-01	Inscription via un numéro de téléphone unique.
AUTH-02	Envoi d'un code OTP par SMS avec validité de 5 minutes géré dans Redis.
AUTH-03	Vérification OTP et activation automatique du compte.
AUTH-04	Attribution du rôle applicatif unifié ROLE_USER et délégation à Keycloak.
AUTH-05	Consultation et mise à jour des informations de profil utilisateur.
AUTH-06	Modération administrative : suspension ou réactivation d'un compte (ROLE_ADMIN).

8.2 MS-Logements

ID	Exigence fonctionnelle
LOG-01	Création, modification, consultation et retrait d'annonces de logements.
LOG-02	Gestion des critères : titre, adresse, ville, quartier, loyer, meublé, chambres et disponibilité.
LOG-03	Téléversement et suppression de photos d'annonces stockées sur MinIO.
LOG-04	Recherche multicritère avec filtres combinés (localisation, budget, type).
LOG-05	Gestion des logements favoris : ajout, suppression et consultation par l'utilisateur.
LOG-06	Modération des annonces de logement par l'administrateur.

8.3 MS-Colocation & Matching

ID	Exigence fonctionnelle
COL-01	Publication d'annonces de recherche de colocation.
COL-02	Consultation et filtrage des annonces de colocation actives.

COL-03	Envoi d'une demande de colocation vers un profil.
COL-04	Acceptation ou refus d'une demande de colocation.
COL-05	Calcul et affichage des suggestions de compatibilité (algorithme Data Science).
COL-06	Émission d'un événement Kafka lors de chaque demande ou acceptation.

8.4 MS-Contrats & Paiements

ID	Exigence fonctionnelle
PAY-01	Génération automatique d'un contrat de bail au format PDF.
PAY-02	Consultation et téléchargement du contrat PDF.
PAY-03	Initialisation d'un paiement en ligne sécurisé (dépôt de garantie, premier loyer).
PAY-04	Validation des transactions via webhook prestataire avec contrôle d'idempotence.
PAY-05	Génération automatique d'un reçu de paiement téléchargeable en PDF.
PAY-06	Historique complet des paiements consultable par le locataire et le propriétaire.

8.5 MS-Communication & Notifications

ID	Exigence fonctionnelle
COM-01	Création et gestion des conversations entre utilisateurs.
COM-02	Envoi et réception de messages en temps réel via WebSocket/STOMP.
COM-03	Consommation des événements Kafka en provenance de tous les microservices.
COM-04	Génération et affichage des notifications applicatives internes.
COM-05	Marquage d'une notification comme lue.
COM-06	Envoi de SMS réels via passerelle externe pour les alertes critiques.

8.6 MS-Avis & Réclamations

ID	Exigence fonctionnelle

REV-01	Dépôt d'une note de 1 à 5 étoiles et d'un commentaire textuel.
REV-02	Calcul et affichage de la note moyenne d'un utilisateur ou logement.
REV-03	Dépôt d'une réclamation liée à une annonce, une colocation ou une prestation.
REV-04	Suivi du statut des réclamations : ouverte, en cours, résolue ou rejetée.
REV-05	Modération des avis et traitement des réclamations par l'administrateur.

8.7 MS-Furniture Marketplace

ID	Exigence fonctionnelle
FUR-01	Publication d'annonces de meubles (vente, don ou échange).
FUR-02	Gestion des photos de meubles hébergées sur MinIO.
FUR-03	Recherche et filtrage des meubles par catégorie, état, prix et localisation.
FUR-04	Envoi d'une demande d'acquisition (achat, don ou échange).
FUR-05	Acceptation d'une demande créant une réservation exclusive.
FUR-06	Clôture de la transaction une fois la remise en main propre effectuée.

8.8 MS-Moving & Transport

ID	Exigence fonctionnelle
MOV-01	Création d'un profil de transporteur (véhicules, zones desservies, tarifs indicatifs).
MOV-02	Dépôt d'une demande de déménagement (adresses, volume, date, options).
MOV-03	Recherche de transporteurs disponibles par zone et notation.
MOV-04	Émission d'un devis chiffré par un transporteur.
MOV-05	Acceptation ou refus d'un devis par le demandeur.
MOV-06	Création de la réservation et suivi des statuts de la course (en cours, terminée).

9. Exigences non fonctionnelles

Contraintes techniques, critères d'isolation et exigences de performance.

Catégorie	Exigences
Architecture	Huit microservices autonomes, principe Database-per-Service (isolation PostgreSQL sans clé étrangère transverse, référence par userId: UUID).
Sécurité	Validation centralisée des jetons JWT par Keycloak sous l'API Gateway, HTTPS, validation stricte des entrées.
Communication asynchrone	Bus d'événements Apache Kafka pour le découplage des traitements et la diffusion des notifications.
Temps réel	WebSocket / STOMP pour la messagerie instantanée avec latence inférieure à une seconde.
Cache & Session	Redis pour le stockage temporaire des codes OTP SMS (TTL 5 minutes) et la présence en ligne.
Stockage de fichiers	MinIO (stockage objet S3) pour toutes les photos (logements, meubles) et documents PDF.
Observabilité	Logs structurés avec correlation ID, endpoints Spring Boot Actuator et métriques de santé.
Déploiement	Conteneurisation intégrale Docker et orchestration locale via Docker Compose.

10. Contrats d'API principaux

Spécifications des principaux endpoints REST et WebSocket pour chacun des huit microservices.

10.1 MS-Utilisateurs & Authentification

Méthode	Endpoint	Description
POST	/api/utilisateurs	Créer un compte en attente de validation OTP
POST	/api/utilisateurs/envoyer-otp	Générer et envoyer un code OTP par SMS
POST	/api/utilisateurs/verifier-otp	Valider le code OTP et activer le compte
GET	/api/utilisateurs/me	Consulter son profil utilisateur
PUT	/api/utilisateurs/me	Mettre à jour ses coordonnées de profil
PATCH	/api/admin/utilisateurs/{id}/statut	Suspendre ou réactiver un utilisateur (admin)

10.2 MS-Logements

Méthode	Endpoint	Description
POST	/api/logements	Publier une nouvelle annonce de logement
GET	/api/logements	Lister les logements actifs avec pagination
GET	/api/logements/{id}	Consulter les détails d'un logement
PUT	/api/logements/{id}	Modifier son annonce de logement
DELETE	/api/logements/{id}	Supprimer ou archiver une annonce
GET	/api/logements/recherche	Recherche multicritère avec filtres
POST	/api/logements/{id}/photos	Téléverser une photo d'annonce vers MinIO
DELETE	/api/logements/{id}/photos/{photoId}	Supprimer une photo d'annonce
POST / DELETE	/api/logements/{id}/favori	Ajouter ou retirer un logement des favoris
GET	/api/logements/favoris	Lister les logements mis en favoris

10.3 MS-Colocation & Matching

Méthode	Endpoint	Description
POST	/api/colocations/annonces	Publier une recherche de colocation
GET	/api/colocations/annonces	Lister les annonces de colocation actives
GET	/api/colocations/annonces/{id}	Détails d'une annonce de colocation
POST	/api/colocations/demandes	Envoyer une demande de mise en relation
GET	/api/colocations/demandes	Lister les demandes envoyées et reçues
PATCH	/api/colocations/demandes/{id}/statut	Accepter ou refuser une demande
GET	/api/colocations/suggestions	Calculer les profils compatibles (Matching DS)

10.4 MS-Contrats & Paiements

Méthode	Endpoint	Description
POST	/api/contrats	Créer les données contractuelles de location
POST	/api/contrats/{id}/generer-pdf	Générer et archiver le contrat de bail PDF
GET	/api/contrats/{id}/document	Télécharger le contrat PDF
POST	/api/paiements	Initier un paiement en ligne (dépôt ou loyer)
GET	/api/paiements/{id}	Consulter le statut d'une transaction
POST	/api/paiements/webhook	Webhook de confirmation du prestataire de paiement
GET	/api/paiements/{id}/recu	Télécharger le reçu de paiement PDF

10.5 MS-Communication & Notifications

Méthode	Endpoint	Description
POST	/api/conversations	Ouvrir une conversation entre utilisateurs
GET	/api/conversations	Lister ses conversations actives

GET	/api/conversations/{id}/messages	Consulter l'historique d'une conversation
POST	/api/conversations/{id}/messages	Envoyer un message (fallback HTTP)
WS	/ws-chat	Point d'entrée WebSocket/STOMP (chat temps réel)
GET	/api/notifications	Lister ses notifications
PATCH	/api/notifications/{id}/lu	Marquer une notification comme lue

10.6 MS-Avis & Réclamations

Méthode	Endpoint	Description
POST	/api/avis	Déposer un avis et une note de 1 à 5
GET	/api/avis	Consulter les avis et note moyenne d'une cible
POST	/api/reclamations	Déposer une réclamation
GET	/api/reclamations	Lister ses réclamations et suivre l'état
PATCH	/api/admin/reclamations/{id}/statut	Traiter et clôturer une réclamation (admin)

10.7 MS-Furniture Marketplace

Méthode	Endpoint	Description
POST	/api/meubles/annonces	Publier une annonce de meuble
GET	/api/meubles/annonces	Lister et rechercher les meubles actifs
GET	/api/meubles/annonces/{id}	Détails d'une annonce de meuble
PUT	/api/meubles/annonces/{id}	Modifier son annonce de meuble
DELETE	/api/meubles/annonces/{id}	Retirer une annonce de meuble
POST	/api/meubles/annonces/{id}/photos	Téléverser des photos vers MinIO
POST	/api/meubles/demandes	Envoyer une demande d'achat/don/échange
PATCH	/api/meubles/demandes/{id}/statut	Accepter une demande (crée la réservation)

GET	/api/meubles/reservations	Lister ses réservations de meubles

10.8 MS-Moving & Transport

Méthode	Endpoint	Description
POST / PUT	/api/transporteurs/me	Créer ou mettre à jour son profil transporteur
GET	/api/transporteurs	Rechercher des transporteurs (zone, note)
POST	/api/demenagements/demandes	Déposer une demande de déménagement
GET	/api/demenagements/demandes	Lister les demandes de sa zone (transporteur)
POST	/api/demenagements/demandes/{id}/devis	Émettre un devis chiffré
PATCH	/api/demenagements/devis/{id}/statut	Accepter ou refuser un devis
GET	/api/demenagements/reservations/{id}	Consulter une réservation et son suivi
PATCH	/api/demenagements/reservations/{id}/statut	Mettre à jour le suivi ou clôturer

11. Stack technique

Technologies et composants d'infrastructure retenus.

Domaine	Technologie retenue
Back-end	Java 17+ / Spring Boot 3.x (Spring Data JPA, Spring Web)
Front-end	React (Single Page Application responsive)
API Gateway	Spring Cloud Gateway (routage, filtrage des requêtes)
Service Registry	Netflix Eureka (découverte dynamique des instances)
Authentification / IAM	Keycloak positionné sous Gateway (OAuth2 / OIDC / jetons JWT)
Bus asynchrone	Apache Kafka (AMQP, topics partitionnés d'événements et alertes)
Messagerie temps réel	WebSocket + STOMP (Spring WebSocket)
Cache & Session	Redis (codes OTP SMS avec TTL 5 min, présence en ligne)
Stockage de fichiers	MinIO (stockage objet compatible S3 pour photos et PDF)
Base de données	PostgreSQL (isolation Database-per-Service, 8 bases dédiées)